

Backend Developer

로그와 데이터로 문제를 정의하고 개선합니다

로그와 데이터를 근거로 문제를 정의하고 개선하는 백엔드 개발자입니다. 백엔드 파트장을 맡아 어뷰징 대응 조직과 일본·글로벌 캐시워크를 담당하며, 운영 관점에서 '왜 그 선택을 했는가'를 고민하는 개발을 지향합니다.

NestJS

Spring

TypeScript

Java

MySQL

Redis

Nginx

AWS

GitHub Actions

Grafana

01 경력 CAREER

2025.05 - 재직 중

정규직 · 백엔드

넛지헬스케어

일본·글로벌 캐시워크 백엔드 담당 · 백엔드 파트장 선임

● 백엔드 파트장 · 2026.04 - 현재

어뷰징 대응 조직 · 일본 캐시워크 · 글로벌 캐시워크 담당

○ 백엔드 개발자 · 2025.05 - 2026.03

일본 캐시워크 프로젝트 백엔드 담당

파트장 업무

- 파트 성과를 1차로 책임 (파트원 3명 · 프로젝트 3개 관리)
- 관리 업무와 개발 실무를 병행
- 일일 스크럼 주도, 업무 배정 · 공수 점검 · 코드 리뷰
- 운영 서비스의 24시간 가용성 관리

개발 업무

- 기능 개발 및 운영
- 어뷰징 탐지 및 분석
- 로그 기반 개선

CASE 01

Nginx 499 에러 대응

4XX 50% 초과 알림의 정체를 로그로 추적

51.3%

요청 4xx 에러

Degraded 알림 시점

499

대부분의 에러 코드

클라이언트 연결 종료

배경

Environment health has transitioned from Warning to Degraded. 51.3% of the requests are erroring with HTTP 4xx.

Beanstalk에서 **4XX 응답이 50%를 초과**했다는 알림이 발생했습니다. 로그를 확인해보니 대부분 **499 에러**였고, 이는 요청 처리 중 **클라이언트가 연결을 끊은 이벤트**를 기록한 것이었습니다.

원인

499 에러는 클라이언트가 연결을 끊어 발생하기 때문에 **서버에서 직접 해결할 수 없었습니다**. 실제로 nginx에는 499로 기록되더라도, 같은 요청이 서버 로그에는 **200 정상 처리**로 남아있었습니다.

요청 타임라인 – 같은 요청이 nginx 499 / 서버 200 으로 갈리는 과정

시간	앱 (클라이언트)	nginx	서버
0ms	요청 전송 →	서버로 요청 포워딩 →	
5ms	즉시 연결 끊음 (응답 안 기다림)	연결 끊김 감지 → 499 기록	처리 시작
50ms			처리 중...
100ms		응답 받을 곳 없어 버림	처리 완료 → 응답 반환

근본 원인

앱이 **백그라운드**로 요청을 보내고 응답을 기다리지 않은 채 연결을 끊으면, nginx는 이를 감지해 **499를 기록**합니다. 앱은 응답을 받지 못해 **재시도**하지만, 재시도 역시 백그라운드 요청이라 또 종료되고 다시 499가 기록됩니다. 이 **재시도 루프**가 누적되어 전체 요청의 50% 이상이 4XX로 집계된 것을 확인했습니다.

후기

– 로그와 알림의 중요성을 깨달았습니다. 에러가 어떻게 발생하는지 파악하는 방법과, 오류 원인을 추적하기 위한 **에러 로깅 설정**을 직접 구성하며 확인할 수 있었습니다.

CASE 02

GA4 기반 DAU 리포팅 개편

서버 집계와 GA4 DAU 불일치를 리포팅 구조 개편으로 해소

배경 데이터팀이 확인하는 **GA4 DAU**와 서버에서 집계하는 **DAU가 상이해**, 데이터 분석 및 리포팅에서 혼선이 생기고 있었습니다. 당시 서버는 DAU를 단순히 리포팅하는 역할만 하고 있었습니다.

해결 **GA4의 활성 사용자 집계 방식**

지정된 기간 동안 사이트/앱에 참여한 순 사용자 수로, 참여 세션이 있거나 아래 이벤트가 수집된 사용자를 활성 사용자로 봅니다.

- 웹: `first_visit` 또는 `engagement_time_msec`
- Android: `first_open` 또는 `engagement_time_msec`
- iOS: `first_open` 또는 `user_engagement`
- 1초 이내에 `user_engagement` 이벤트가 감지되면 활성 사용자로 간주

반면 서버는 **특정 API가 호출된 고유 유저 수**를 카운팅하고 있었습니다. 서버 집계 로직을 고도화해 맞출 수도 있었지만, **서버 집계를 없애고 GA4 데이터를 서버에서 리포팅으로 가져오는 방식**으로 변경했습니다. 직접 집계하는 것보다, 실제로 DAU를 사용하는 팀이 활용하는 데이터를 리포팅하는 방향이 더 낫다고 판단했기 때문입니다.

자연 이슈 GA4는 **데이터 수집 지연**이 있어, 리포트를 요청하는 시점에 따라 DAU 값이 크게 달라졌습니다. 그래서 당일 데이터만 리포팅하지 않고 **전일 DAU도 함께 리포팅**해 값의 변화를 확인할 수 있게 했고, 데이터를 불러올 때 **최근 일주일치를 함께 갱신**해 최신화된 DAU를 볼 수 있도록 했습니다.

후기 — 예전에는 로직이 제대로 동작하는지에만 집중했습니다. 이번 작업으로 **서비스 운영 관점에서 어떤 데이터를 제공·수집해야 하는지, 그 데이터가 어떤 의미를 갖는지**를 고민하게 됐고, Slack·Google Analytics 연동으로 필요한 데이터를 원하는 형태로 수집·활용해 운영에 기여할 수 있었습니다.

CASE 03

업무 요청 방식 개선

슬랙으로 흩어지던 업무 요청을 Notion 작업 리스트로 일원화 · 글로벌 캐시워크 도입

문제 기존에는 업무를 **슬랙으로 요청**하면 백엔드 내부에서 확인해 대응하고, 우선순위는 기획에 요청해 일정을 조정·처리하는 방식이었습니다. 그러다 보니 **누가 어떤 업무를 하고 있는지** 보이지 않았고, 어디에 병목이 있는지·무엇을 먼저 해야 하는지 일일이 파악하기 어려웠습니다. 코드 착수 전의 업무 파악·일정 산정 단계는 관리 밖에 있었습니다.

개선 **Notion 작업 리스트 카드 기반으로 요청 체계를 정립**

- **요청 방법** — 요청자가 작업 리스트에서 직접 카드를 생성(기본 템플릿에 맞춰 기재)
- **요청 시점** — 슬랙에 최초 공유하는 시점에 Notion 카드도 함께 생성해 **코드 착수 전부터 관리**
- **기한 체계** — 검토 기한과 마감 기한을 분리하고, 승인권자 컨펌 후 마감 기한을 확정

두 기한의 정의 — 우선순위 파악이 목적

구분	의미
검토 기한	백엔드가 업무를 파악하고 일정 산정을 마치기를 희망하는 날짜
마감 기한	개발 완료(테스트 서버 반영 → API 연동·QA 가능 상태)를 희망하는 날짜

1시간 이내의 간단한 업무는 별도 컨펌 없이 진행하고, CS·이슈 파악처럼 일정을 예측하기 어려운 경우엔 검토 기한만 먼저 기재한 뒤 검토 후 필요한 시점에 마감 기한 컨펌을 요청하도록 했습니다.

기대 효과

- **작업 리스트 활용도 증가** — 기존엔 백엔드 일정을 파트장이 관리·전달했지만, 이제 요청자가 백엔드 업무 현황을 직접 확인하고 일정·우선순위 판단에 참고
- **검토 단계의 업무 관리 강화** — 요청 시점을 앞당겨, 코드 수정 이전의 업무 파악·일정 산정에 투입되는 리소스까지 가시화·관리

후기 — 개발은 코드 작업만이 아니라 **그 앞단의 요청·검토·일정 산정까지가 관리 대상**이라는 걸 체감했습니다. 이 방식을 **글로벌 캐시워크에 도입**해, 흩어진 요청을 한곳에서 추적하고 우선순위를 함께 합의하는 흐름으로 운영하고 있습니다.

CASE 04

어뷰징 탐지용 사용자 클러스터링

계정 간 관계를 Union-Find로 묶어 다계정 어뷰징 탐지

30,000+

어뷰징 차단

누적 차단 계정

배경 다계정 생성을 통한 리워드 부정 적립 어뷰징을 탐지해야 했습니다. 의심되는 계정들을 하나의 집단으로 묶어, 주기적으로 클러스터를 업데이트하며 어뷰징 의심 집단을 블락하는 시스템이 필요했습니다.

의사결정 1 계정 간 관계 클러스터링을 Union-Find로 구현
계정 간 관계는 **그래프 자체의 방향성이 필요 없었습니다**. "같은 집단인지"와 "집단의 인원 수"만 파악하면 됐기에 다음 이유로 **Union-Find**를 선택했습니다.

- **Graph DB** — 별도 인프라 추가가 필요
- **BFS/DFS** — 전체 관계 테이블과 인접 리스트를 매번 로드해야 하고, 메모리에 들고 있기도 애매
- **재귀 CTE** — MySQL 8.0의 재귀 깊이 제한·성능 문제가 있고, 일본 캐시워크 특성상 DB 쿼리가 길어지면 connection이 부족해 다른 요청이 오래 대기할 수 있음

자료구조 비교 — 증분 업데이트와 DB 부하 관점

	Union-Find	BFS/DFS	Graph DB	재귀 CTE
증분 업데이트	$\approx 0(1)/\text{edge}$	불가	재계산 필요	불가
전체 재계산	$\approx 0(N)$	$O(N+E)$	$O(N+E)$	$O(N \cdot E)$
DB 쿼리(증분)	$2R + 1 \text{ batch } W$	N/A	$1W + 1R$	N/A
메모리	$O(N)$	$O(N+E)$	DB 내부	DB 내부

의사결정 2 증분 적재 + 배치 처리

- 직전 시간에 새로 발생한 계정 간 관계를 클러스터에 **증분 적재**
- 크기가 **일정 규모 이상인 클러스터**에 대해 검증 및 처리(블락)

실시간 증분 적재를 하지 않는 이유 ① 동시성 문제 — 거의 동시에 들어오는 관계는 커밋 타이밍에 따라 하나여야 할 클러스터가 잘못 분리될 수 있습니다.

클러스터 분리 시나리오

TEXT

실제: A→B, C→B, D→C (거의 동시)

기대: {A, B, C, D} 하나의 클러스터

실제 발생 가능:

T1: A→B 처리 → {A, B}

T2: C→B 처리 → {A, B, C}

T3: D→C 처리 → C 조회 시 T2 미커밋 → {D, C} 별도 생성

결과: {A,B,C}와 {D,C}로 분리되거나 userId 중복 에러

② 외부 API 의존 — 블락 판단에 외부 API 호출·처리가 필요해 실시간 블락에는 적합하지 않습니다.

트러블슈팅 배치 재구축의 **역등성(determinism) 확보**
cron은 매 실행마다 기존 클러스터를 메모리에서 처음부터 다시 구축합니다. 그런데 두 트리의 **rank가 같을 때 루트를 정하는 규칙이 비결정적**이라, 같은 데이터를 넣어도 실행할 때마다 루트(clusterId)가 바뀌는 문제가 있었습니다. clusterId가 흔들리면 차단 이력 등 후속 데이터와의 매칭이 깨집니다.

- union의 **tie-break**를 **결정적으로 고정**해, 같은 입력이면 항상 같은 clusterId가 나오도록 **역등성**을 확보
- 두 클러스터를 합칠 때 한쪽이라도 차단 상태면 합쳐진 클러스터 전체로 **차단 여부를 전파**

후기 — 알고리즘을 문제 풀이에서만 다뤄봤지, **실제 서비스 코드에 직접 적용한 것은 이번이 처음**이었습니다. Union-Find 하나를 고르는 데에도 인프라 비용·DB 커넥션·동시성·배치 역등성처럼 고려할 것이 훨씬 많았고, **운영 환경과 제약을 함께 고민하며 설계하는 과정**이 새롭고 즐거웠습니다.

CASE 05

매월 1일 데드락·메모리 스파이크 추적

몰리는 시점만 같고 원인은 별개 — 근본 원인을 코드의 비결정적 락 순서로 특정

- 증상

운영 DB에서 **FreeableMemory** 급락·스왑·DB 커넥션 급증 등 메모리 이슈가 관측됐습니다. 추이를 보니 평소엔 멀쩡하다가 **매월 1일 오전에만** 발생하는 패턴이었고, 다른 지표를 훑어보니 **데드락도 똑같이 1일에만 스파이크**치고 있었습니다.
- 가설

1일에만 몰린다 → 월간 크론이 도는 건가?
같은 날 두 지표가 함께 튀니, **1일에 도는 무거운 월간 배치(크론)**가 부하를 주는 게 아닐까 의심했습니다. 스케줄 설정이 **AWS EventBridge**에 흩어져 있고 다른 리전 가능성까지 있어 콘솔로 눈으로 확인하긴 어려워, **AI로 AWS CLI 명령**(`aws events list-rules` · `aws scheduler list-schedules`)을 구성해 전 리전 룰·스케줄을 한 번에 훑었습니다. → **월간 크론은 없음**.
- 추적

크론이 원인이 아니니, 데드락 본문과 부하를 일으키는 로직을 직접 들여다봤습니다. 데드락 본문 로깅이 꺼져 있고 에러 로그도 export되지 않아, `SHOW ENGINE INNODB STATUS` 의 `LATEST DETECTED DEADLOCK` 으로 직전 데드락 본문을 확보했습니다. 함께 확인한 로직은 **내부 정책상 월 단위로 끊겨, 1일이면 사실상 전 유저가 대상이 되는 구조**였습니다. 결국 두 지표는 **1일에 몰린 시점만 같을 뿐 원인은 서로 별개**였습니다.
- 원인 ①

메모리 이슈 — 월 단위 정책이 1일에 집중
메모리 압박의 원인은 이 로직이었습니다. 월 단위 정책이라 1일에 전 유저 대상 처리가 인증·로그인 시점마다 일제히 실행되며 부하가 몰렸고, 오전 트래픽 대비 scale-out과 맞물려 writer로 커넥션이 집중된 것이 증폭 요인이었습니다.
- 원인 ②

데드락 — 부하가 아니라 코드의 비결정적 락 순서
반면 데드락의 원인은 메모리·부하가 아니라 코드였습니다. 해당 로직이 두 테이블 A·B를 `Promise.all` 로 **동시에 갱신**해, 한 트랜잭션 안에서 **락 획득 순서가 보장되지 않았습니다**. 두 트랜잭션이 엇갈린 순서로 같은 두 행을 잠그면 서로 상대의 락을 기다려 교착이 발생합니다. **1일에만 보이는 건 같은 로직이 1일에 대량 실행되며 두 트랜잭션이 겹칠 확률이 올라가 교착이 표면화**됐기 때문입니다. 즉 부하는 **발생 확률을 끌어올린 방아쇠일 뿐**, 근본 원인은 코드의 비결정적 락 순서였습니다.

데드락 시나리오	TEXT
T1: A, B 를 Promise.all 로 갱신	
T2: 같은 두 행을 Promise.all 로 갱신 (순서 비보장)	
T1 — A 락 획득 → B 락 대기	
T2 — B 락 획득 → A 락 대기	
결과: 서로 상대 락을 기다리는 교착 → victim 트랜잭션 롤백	

영향은 1일 오전 한정으로 건수는 적었고, 한쪽 흐름은 다음 접속 시 재시도로 자가치유되나 **다른 쪽은 처리 누락 가능성**이 있었습니다.

후기 — 여러 지표를 교차로 확인하며 원인을 좁혀가는 과정이 재미있었습니다. 동시에 **로깅의 중요성**을 절감했는데, 데드락 본문 로깅이 꺼져 있어 `SHOW ENGINE INNODB STATUS` 로 직접 본문을 떼야 했고, 로깅이 제대로 갖춰져 있었다면 원인 파악이 훨씬 빨랐을 것이라 느꼈습니다. 또한 여러 리전에 흩어진 설정과 지표를 **AI로 한 번에 훑어** 사람이 일일이 뒤지는 것보다 훨씬 빠르게 패턴을 짚어낼 수 있었던 점도 인상적이었습니다.

fancamp 부스트캠프 그룹 프로젝트 · FE 2 · BE 2

기간 2023.10 — 2023.12

담당 DB 속도 개선 · 모니터링 기반 성능 개선 · 협업 환경 개선

GITHUB boostcampwm2023/web02-fancamp

- NestJS
- React
- MySQL
- Nginx
- GitHub Actions
- Grafana
- Prometheus

CASE 1-1

부하테스트 중 성능 개선 모니터링

상황

K6 부하테스트 중 평균 응답 17.5초, 상위 5%는 거의 1분에 달했습니다. Grafana·Prometheus로 관측 환경을 갖추고, CPU 80% 상황에서 멀티스레딩(PM2 cluster)을 1·2·4 스레드로 비교했습니다.

스레드 구성별 부하테스트 결과

	싱글	cluster 2	cluster 4
CPU 사용률	80%	60%	40%
응답시간	17s	4.79s	5.23s
실패율	7%	40%	40%
성공 요청 수	22,661	72,616	64,854
event loop lag	10ms	50ms	80ms

cluster로 응답 시간은 단축됐지만 실패율과 event loop lag이 오히려 증가했습니다. 웹 서비스에서는 응답 성공률이 더 중요하다고 판단해 싱글 스레드를 유지했습니다.

CASE 1-2

MySQL vs MongoDB 입출력 비교

검증

입출력이 잦은 채팅 테이블을 NoSQL로 바꾸면 빨라질지 검증했습니다.

페이지네이션 크기별 조회 시간

크기	MySQL	MongoDB	차이
20개	457.7ms	448.3ms	9.3ms
1,000개	713.8ms	566.5ms	147.3ms
10,000개	1036.3ms	641.8ms	394.5ms

크기가 커질수록 MongoDB가 빨라지지만 실제 페이지네이션은 20개 단위라 차이가 미미했고, 사용자 정보 변경 시 NoSQL은 전체 채팅 수정이 필요해 일관성 관리가 쉬운 MySQL을 유지했습니다.

CASE 1-3

다중 테이블 동시 접근 속도 개선

-98.9%

조회 실행시간

46ms → 0.503ms

31%

1차 개선 (map)

Join 대비

개선

피드형 게시글 조회 시 글·작성자·댓글을 함께 가져오며 **Join 다발 + N+1** 쿼리가 발생했습니다. **EXPLAIN ANALYZE** 로 4가지 방식을 비교했습니다.

조회 방식별 실행 시간

방식	실행 시간
Join	223ms
인덱스 추가	46ms
chat→user 조인	2.59ms
chat→user IN	0.503ms

N+1 회피와 쿼리 요청 수 감소가 더 중요하다고 판단해 DB의 IN 절 방식을 선택했습니다.

CASE 1-4

Notion 기반 협업 개선

개선

프론트-백엔드 협업 중 API 수정 요청 누락·중복 혼선이 있어 '콜센터' **체크리스트**를 도입했습니다. Jira 이슈 번호(**B6P-214**)를 붙여 위에서부터 순서대로 처리하고 Slack에 한 줄로 공유했습니다.

- 요청 반복·혼선이 줄어 작업 속도 향상
- 요청이 기록으로 남아 재확인이 수월
- 비대면·시차 환경에서도 콜센터만 보고 작업 진행

kopilot 한국어 글쓰기 교정 · 풀스택 3

기간 2024.07 — 2024.08

담당 실시간 맞춤법 검사 · 피드백 프롬프트 · 브라우저 저장

GITHUB kopilot2024/kopilot

- NestJS
- JavaScript
- MySQL
- Redis
- Nginx
- Clova AI

CASE 2-1

쓰로틀링 기반 실시간 맞춤법 검사

설계

실시간 첨삭을 위해 매 입력마다 요청을 보내니 **문장 당 100번 이상** 요청이 발생해 API가 거부됐습니다. 디바운싱은 실시간성과 맞지 않아 **쓰로틀링**으로 정해진 간격마다 요청을 보냈습니다.

- 디바운싱 — 입력이 끝나야 요청 → 실시간성과 맞지 않음
- 쓰로틀링 — 정해진 간격마다 요청 → 실시간성 보장
- 메시지 큐·토큰 방식도 검토했으나, 요청 제한·비용이 없어 쓰로틀링으로 많은 요청을 보내는 방향 선택

 요청 수를 **문장 당 약 20번(80% 감소)**으로 줄여 API 오류 없이 실시간 검사 실행

브라우저 저장 방식 설계

회원 기능 대신 **브라우저 단 저장**으로 글·피드백을 보관하며 용도에 따라 두 저장소를 나눴습니다.

- ME:RROR 폴스택 2 · AI 5

담당 Spring 대시보드 · Template Engine 프론트 · Redis 사고 통계

Keras

Redis Sorted Set 기반 통계

최근 6개월 사고 내역을 분석해 **사고 원인별 비율**을 계산하고, 이를 가중치로 감점을 산정해야 했습니다. 날짜를 score로 둔 **Sorted Set**으로 효율적인 범위 쿼리를 구성했습니다.

- zrange** 후 개별 데이터를 for 루프로 조회하니 **20초**가 소요됐습니다. Redis **Pipeline**으로 왕복을 묶어 해결했습니다.

Pipeline 적용으로 20초 → 200ms, 약 100배 성능 향상

04 경험 · 학력 EXPERIENCE & EDUCATION

경험 · 활동

- 2023.07
– 2023.12
- 네이버 부스트캠프 (수료)**
- Vanilla JavaScript로 웹 개발 학습·경험
 - NestJS·TypeScript 기반 WebSocket 실시간 통신 서비스 구현
- 2024.09
– 2025.02
- KT Aivle School**
- AI 학습·활용 방향을 다시 익힘
 - 프로젝트로 AI 활용 및 Spring 경험
- 2022.03
– 2023.03
- 생명정보학 학부 연구생**
- **Blast**: miRNA를 Seq2seq·Attention으로 예측
 - **GNN_DOVE**: 단백질 구조를 GNN으로 예측

학력

- 2018.03
– 2024.02
- 홍익대학교 컴퓨터공학과**
- 학사 졸업
- 졸업 프로젝트 "오늘 뭐 먹지" (2023.03~11)
 - 네이버 리뷰 크롤링 → 감정 분석·Regression으로 별점 예측